

AntiGravity — Estrutura & Módulos

Módulos do Sistema · Arquivos · Tech Stack · Schema de Dados · Evolução MVP → SaaS

■ Módulos do Sistema — Status de Implementação

■ Módulo 01 — Interface Web (UI)

■ Feito	Drag & drop para upload de .zip
■ Feito	Campo "Nome do Caso"
■ Feito	Seletor de modelo Whisper (velocidade/precisão)
■ Feito	Terminal verde de progresso em tempo real
■ Próximo	Seletor de período (hoje/semana/custom)
■ Próximo	Exibição de QR Code PIX
■ Futuro	Visualização da cronologia no browser
■ Futuro	Design responsivo (mobile)

■ ■ Módulo 02 — Parser de Conversas

■ Feito	Regex para detectar mensagens de texto
■ Feito	Identificação de remetentes
■ Feito	Extração de nomes de arquivos de áudio
■ Feito	Parsing de datas no formato BR e EN
■ Próximo	Filtro por período selecionado
■ Feito	Suporte a grupos (múltiplos remetentes)
■ Feito	Deteccção de mensagens apagadas
■ Futuro	Suporte a stickers e outros mídias

■ Módulo 03 — Motor de Transcrição IA

■ Feito	Conversão .opus → .wav (imageio-ffmpeg)
■ Feito	Whisper local (modelo tiny/base/small/medium)
■ Feito	Auto-deteccção de idioma
■ Feito	Transcrição em lote (todos os áudios)
■ Próximo	Integração Groq API (cloud rápido)
■ Próximo	Integração Whisper API OpenAI (fallback)
■ Futuro	Resumo executivo com LLM (gpt-4o-mini)
■ Futuro	Identificação de speakers (diarization)

■ Módulo 04 — Gateway de Pagamento

■ Feito	Contagem de áudios no período (parcial)
■ Próximo	Cálculo do valor (qtd × R\$2,00)
■ Próximo	Criação de cobrança PIX dinâmico
■ Próximo	Exibição de QR Code na interface
■ Próximo	Webhook de confirmação de pagamento
■ Próximo	Token de session vinculado ao pagamento
■ Futuro	Planos mensais com créditos
■ Futuro	Histórico de cobranças

■ Módulo 05 — Consolidação e Output

■ Feito	Merge cronológico texto + áudio transcrito
■ Feito	Exportação em .txt formatado
■ Feito	Cabeçalho com metadados do caso
■ Feito	Abertura automática da pasta no Finder
■ Próximo	Exportação em PDF com layout profissional
■ Próximo	Link de download temporário (cloud)
■ Futuro	Visualização web da cronologia
■ Futuro	Formato jurídico padronizado

■ Módulo 06 — Segurança & LGPD

■ Feito	Arquivos processados apenas em memória temporária
■ Feito	Nenhum envio a APIs externas (fase local)
■ Próximo	Deleção automática pós-download
■ Próximo	HTTPS obrigatório no deploy cloud
■ Próximo	Token de download de uso único
■ Próximo	Sem log de conteúdo transcrito
■ Futuro	Política de privacidade na interface
■ Futuro	Cron de limpeza de arquivos órfãos

■ Estrutura de Arquivos do Projeto

whatsapp-lucas-felipe-whisper/	← Raiz do projeto
whatsapp_local.py	← Script original CLI (v1)
whatsapp_pipeline.py	← Pipeline principal de processamento
Start_AntiGravity.command	← Launcher macOS (duplo clique)
CONTEXT.md	← Documentação do projeto
web_ui/	← Interface Web Flask
app.py	← Servidor Flask principal
templates/	
index.html	← UI principal (drag & drop)
static/	
styles.css	← Estilos da interface
main.js	← JavaScript da interface
uploads/	← Arquivos temporários de upload
casos/	← Output por caso processado
{NOME_DO_CASO}/	
01_zip/	← Arquivo .zip original
02_extraido/	← Conteúdo extraído
03_audios/	← Áudios .opus isolados
04_transcricoes/	← .txt por áudio transcrito
05_consolidado/	← ■ Relatório final aqui
TRANSCRICAO_{caso}_{data}.txt	← Arquivo de saída final
ideias/	← Documentos de planejamento
antigravity-01-infraestrutura-custos.pdf	← Este documento
antigravity-02-fluxograma.pdf	
antigravity-03-estrutura-modulos.pdf	

■ Tecnologias em Detalhe

■ Python + Flask

Backend principal. Flask serve como servidor web leve para a UI local, enquanto scripts Python orquestram todo o pipeline.

■ Usado em: app.py, whatsapp_local.py, whatsapp_pipeline.py

```
@app.route("/processar", methods=["POST"]) def processar(): zip_file = request.files["arquivo"] caso = request.form["nome_caso"] return pipeline.run(zip_file, caso)
```

■ OpenAI Whisper (Local)

Motor de transcrição open-source da OpenAI. Roda completamente na máquina local. Suporta múltiplos modelos: tiny (mais rápido) → small → medium → large (mais preciso).

■ Sem custo de API — usa o hardware do seu Mac

```
import whisper model = whisper.load_model("small") result = model.transcribe("audio.wav", language="pt", fp16=False) texto = result["text"]
```

■ imageio-ffmpeg

Biblioteca Python com FFmpeg portátil embutido. Resolve o problema de dependência do sistema — sem precisar instalar ffmpeg via Homebrew.

■ Converte .opus (WhatsApp) → .wav (16kHz mono) para o Whisper

```
import imageio_ffmpeg ffmpeg_path = imageio_ffmpeg.get_ffmpeg_exe() os.environ["PATH"] += ":" + os.path.dirname(ffmpeg_path) # Agora whisper.transcribe() funciona!
```

■ Groq API (Próximo)

API de inferência ultrarrápida com modelos open-source. Suporta Whisper large v3 com velocidade ~10x superior à OpenAI, com free tier generoso.

■ Caso de uso: versão cloud onde o usuário paga via PIX remotamente

```
from groq import Groq client = Groq(api_key="...") transcription = client.audio.transcriptions.create(file=audio_file, model="whisper-large-v3", language="pt" )
```

■ Asaas PIX API (Próximo)

Gateway brasileiro com PIX dinâmico. Taxa de R\$0,01 fixo por transação — o mais barato para tickets de R\$2,00.

■ Fluxo: criar cobrança → QR Code → webhook pix_paid → liberar processamento

```
# POST /api/v3/charges payload = { "billingType": "PIX", "value": 2.00, "dueDate": today, "externalReference": session_id }
```

■ Supabase (Futuro)

Backend as a service para a versão SaaS. PostgreSQL, autenticação, storage e realtime sem gerenciar servidor.

■ Uso futuro: histórico de casos, créditos do usuário, logs de transações

```
from supabase import create_client supabase = create_client(URL, KEY) supabase.table("casos").insert({"user_id": uid, "nome": caso, "audios": qtd }).execute()
```

Schema de Dados — Versão Cloud (Supabase PostgreSQL)

Na versão atual (local), não há banco de dados. O schema abaixo representa a estrutura futura para a versão SaaS com Supabase PostgreSQL.

Campo	Tipo	Descrição
id (PK)	uuid	Chave primária — Supabase Auth
email	text	Email de contato do usuário
creditos	integer	Créditos de áudio disponíveis
plano	text	free / payg / mensal
criado_em	timestamptz	Data de cadastro

Tabela: users — Usuários da plataforma

Campo	Tipo	Descrição
id (PK)	uuid	Chave primária
user_id (FK)	uuid	FK → users.id
nome_caso	text	Nome dado pelo usuário ao caso
qtd_audios	integer	Quantidade de áudios transcritos
modelo_whisper	text	Modelo usado: tiny/small/medium
status	text	pending / processing / done
download_token	text	Token único de 1 uso para download
criado_em	timestamptz	Data e hora do processamento

Tabela: casos — Transcrições processadas

Campo	Tipo	Descrição
id (PK)	uuid	Chave primária
caso_id (FK)	uuid	FK → casos.id
gateway_txid	text	ID da transação no gateway (MP/Asaas)
valor	numeric	Valor cobrado em R\$
status	text	pending / paid / expired
pago_em	timestamptz	Timestamp da confirmação do pagamento

Tabela: pagamentos — Cobranças PIX

Evolução: MVP Local → SaaS Completo

Funcionalidade	v0 — Local Atual	v1 — MVP Web	v2 — SaaS
Interface	Web UI local (Flask)	Web pública + domínio	App responsivo + PWA
Transcrição	Whisper small local	Groq API (cloud)	Multi-engine + diarização

Pagamento	Nenhum	PIX via Asaas	PIX + Cartão + Assinatura
Multi-usuário	Não	Sim (sem login)	Login + histórico
Output	.txt local	.txt + link download	PDF jurídico + web view
Filtro período	Nenhum (tudo)	Hoje/Semana/Custom	Qualquer range + busca
Banco de dados	Nenhum	Nenhum	Supabase PostgreSQL
Segurança	Alta (100% local)	Boa (HTTPS + token)	Máxima (auth + RLS)
Deploy	macOS .command	Railway / Render	VPS + Docker + CDN
Esforço de dev	0 (pronto)	~3 semanas	~2-3 meses

■ Estratégia: Manter a v0 para uso próprio enquanto desenvolve a v1. Com apenas 35 transcrições/mês já paga toda a infra. Foque primeiro no mercado jurídico — advogados têm alta demanda por transcrições de evidências de WhatsApp.